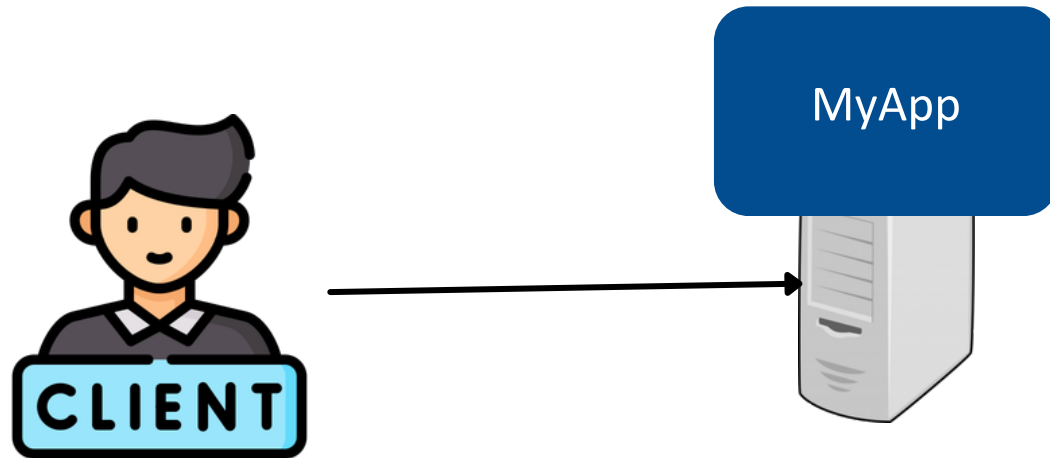


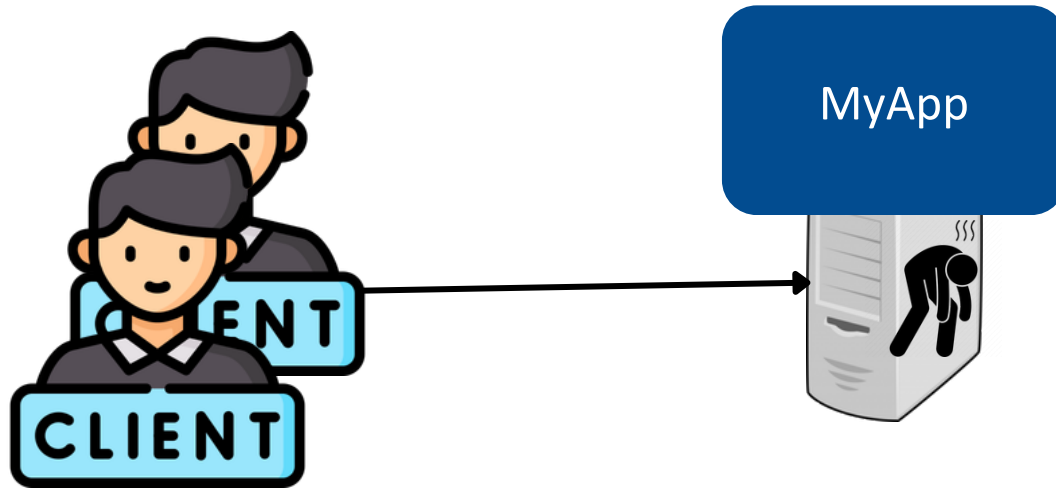
Monólitos vs Micro-Serviços

Um case de sucesso

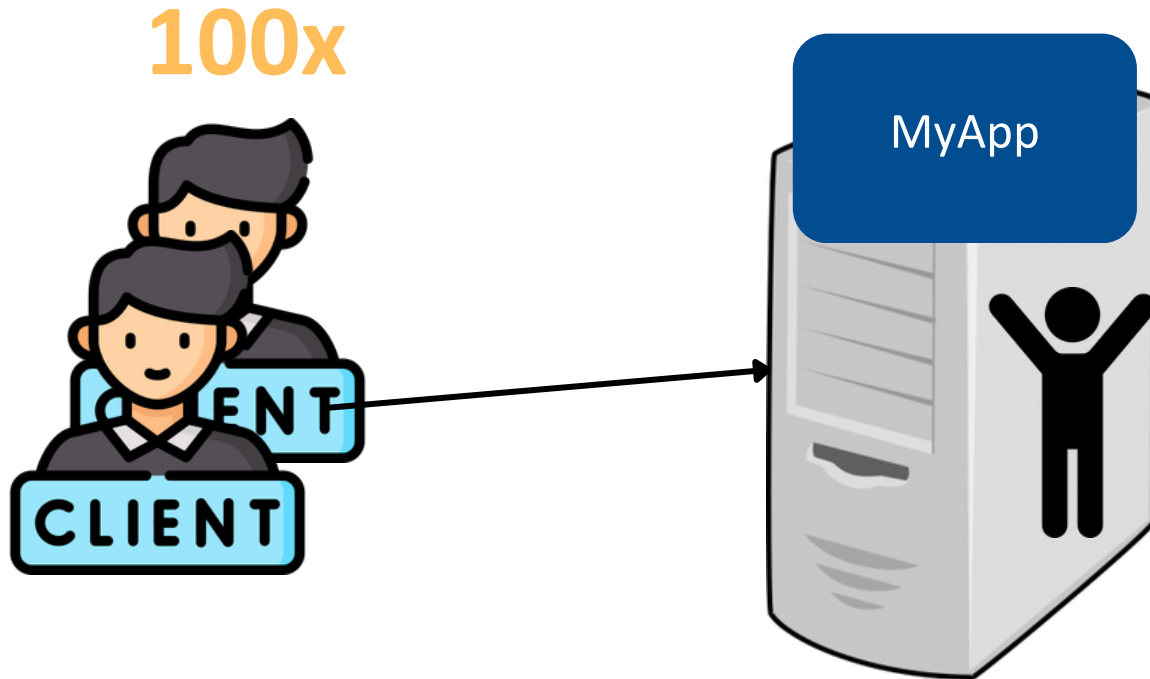


Um case de sucesso

100x



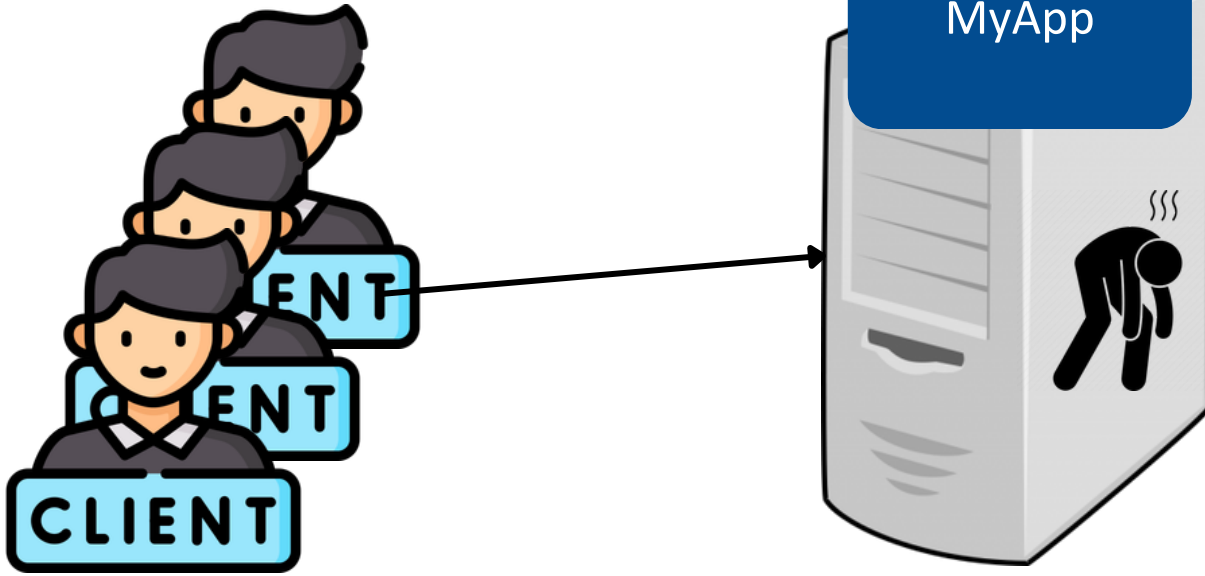
Um case de sucesso



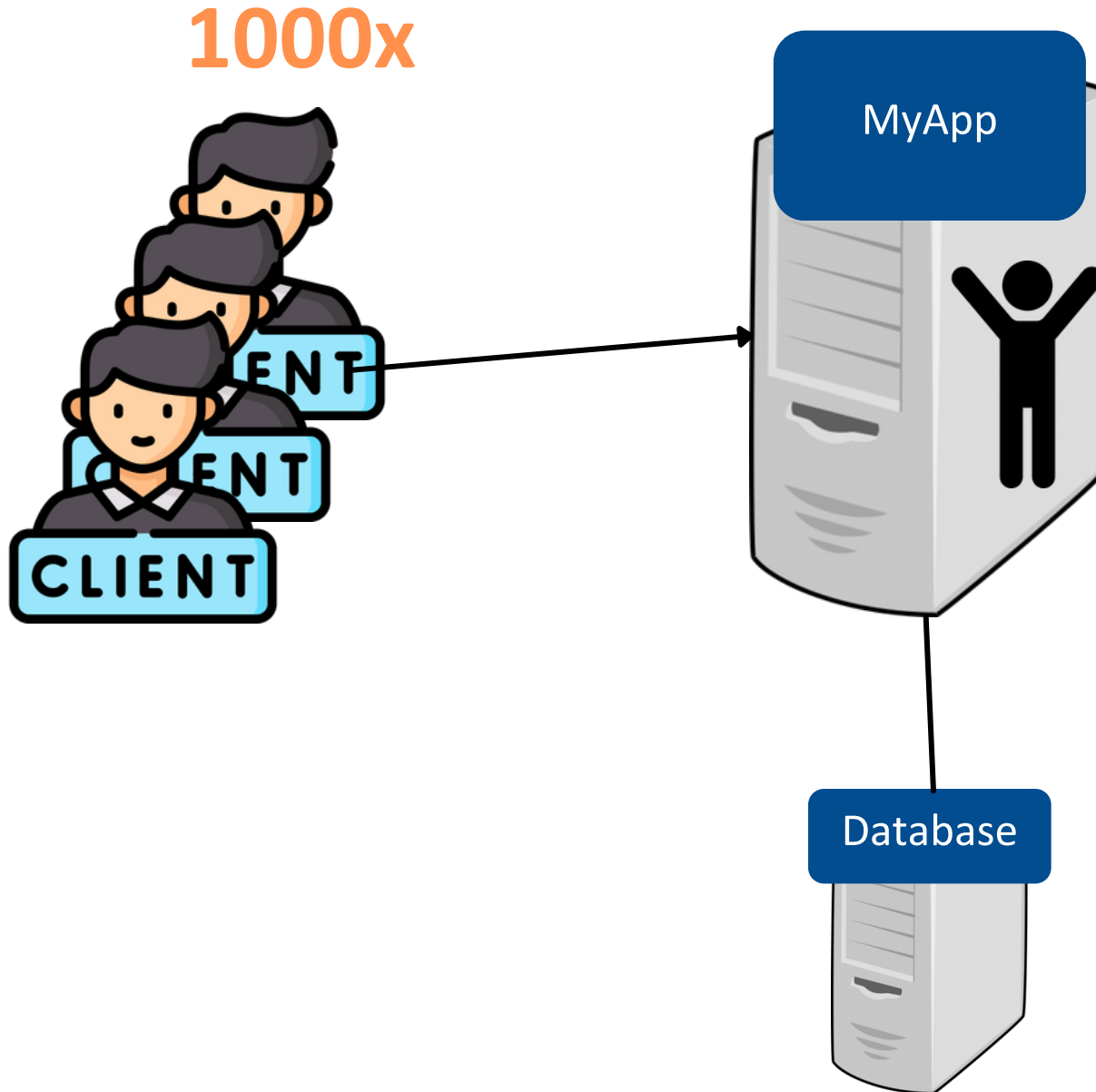
Scaling vertical com uma máquina mais potente!

Um case de sucesso

1000x



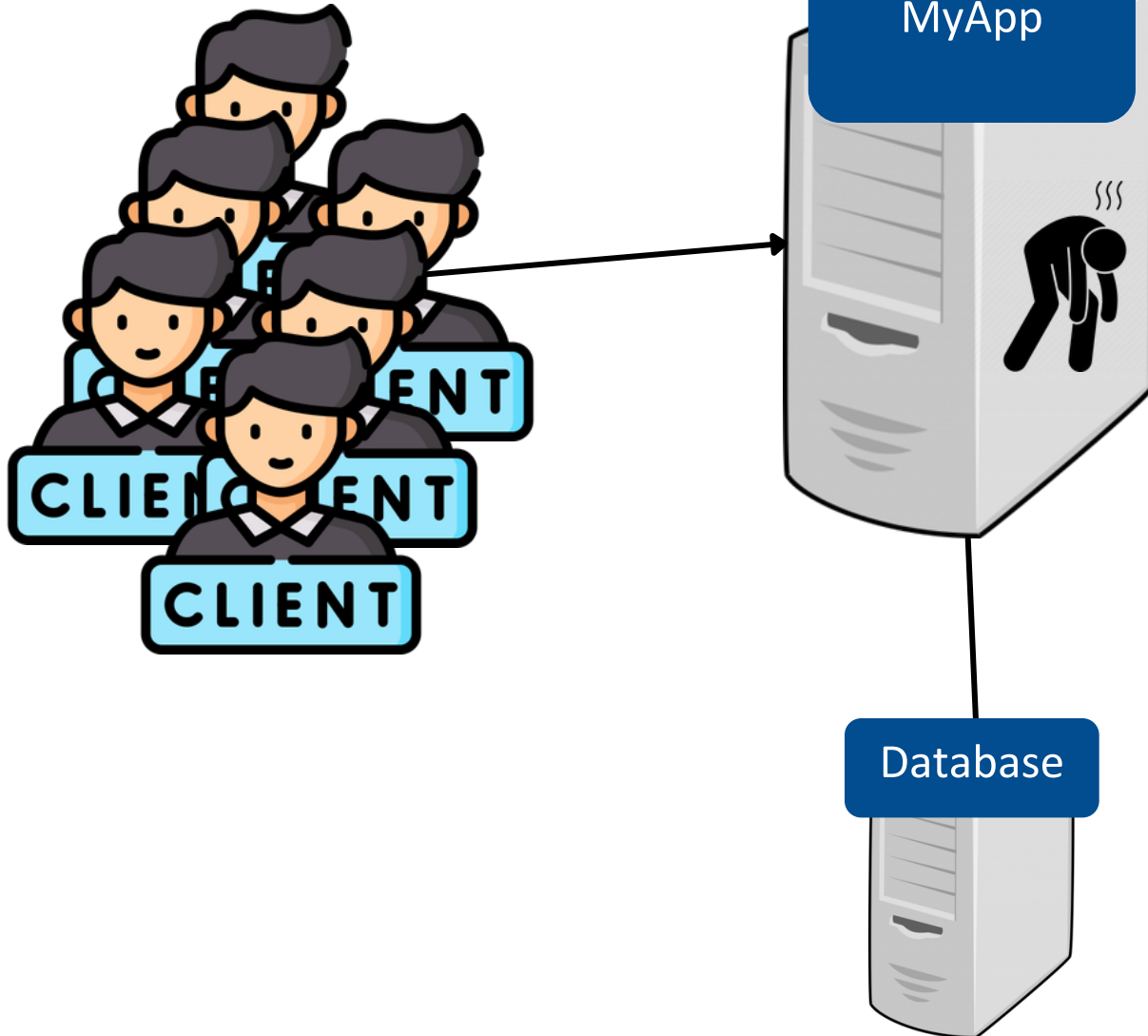
Um case de sucesso



+1 Tier

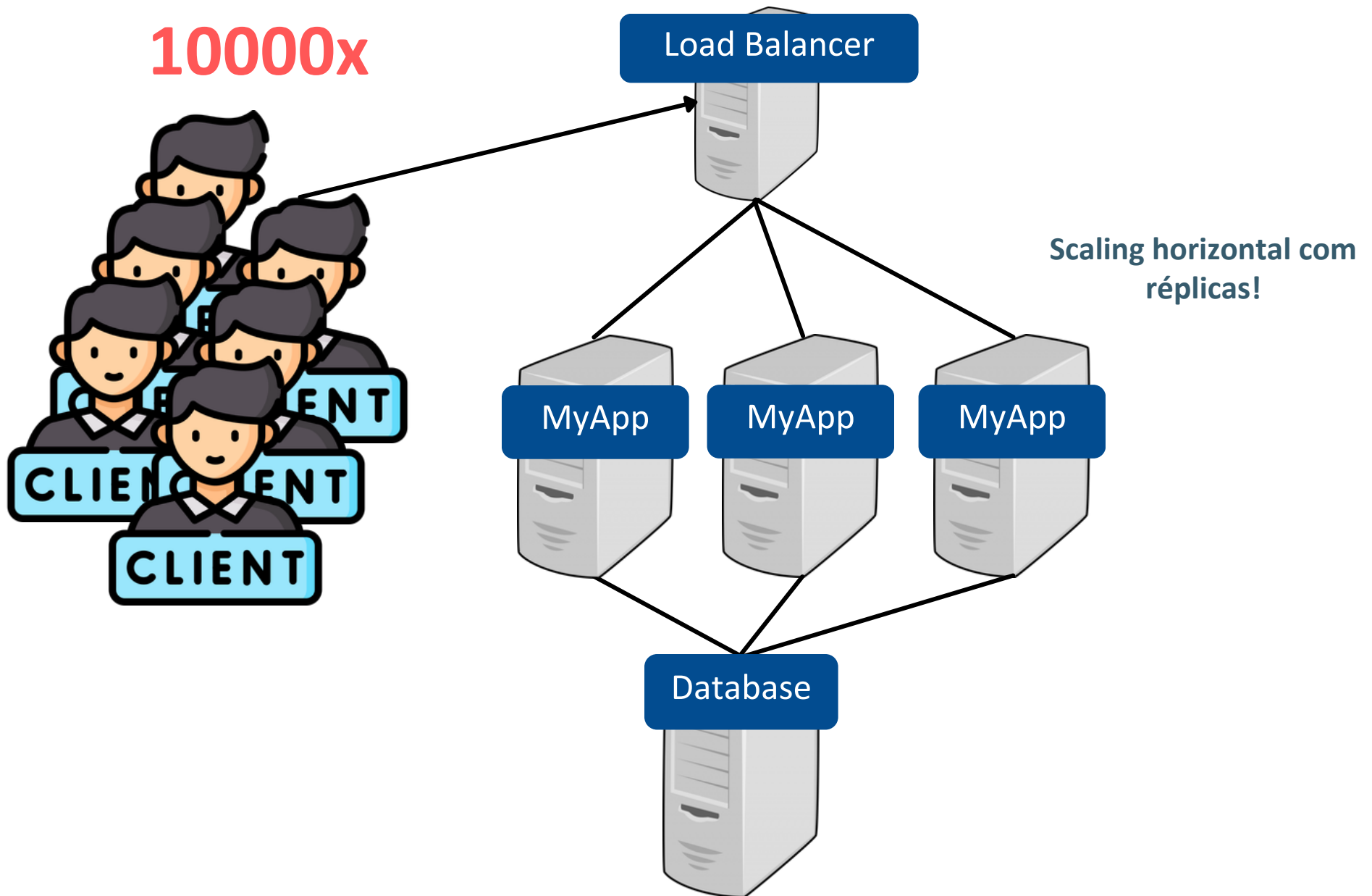
Um case de sucesso

10000x



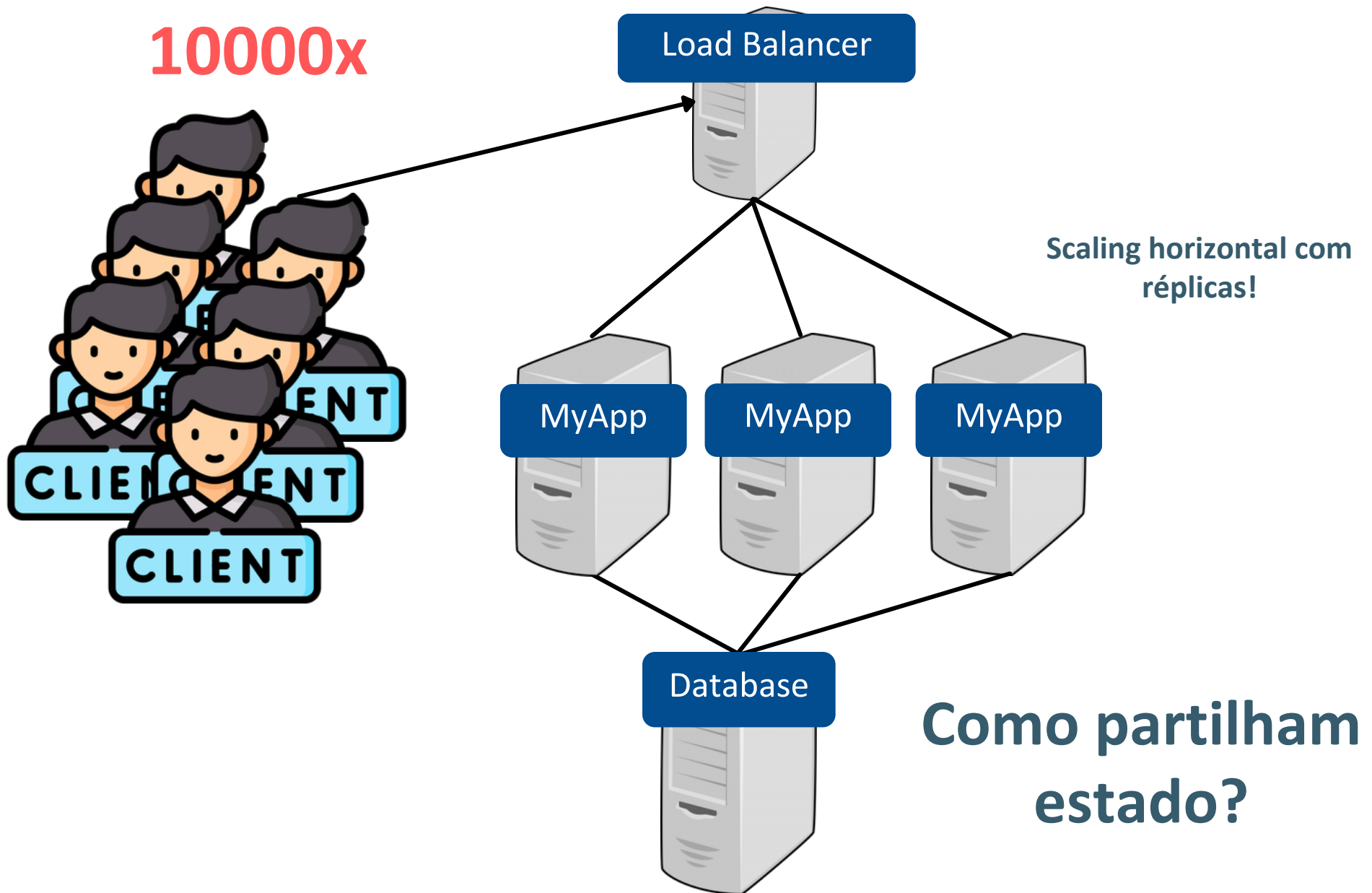
Um case de sucesso

10000x



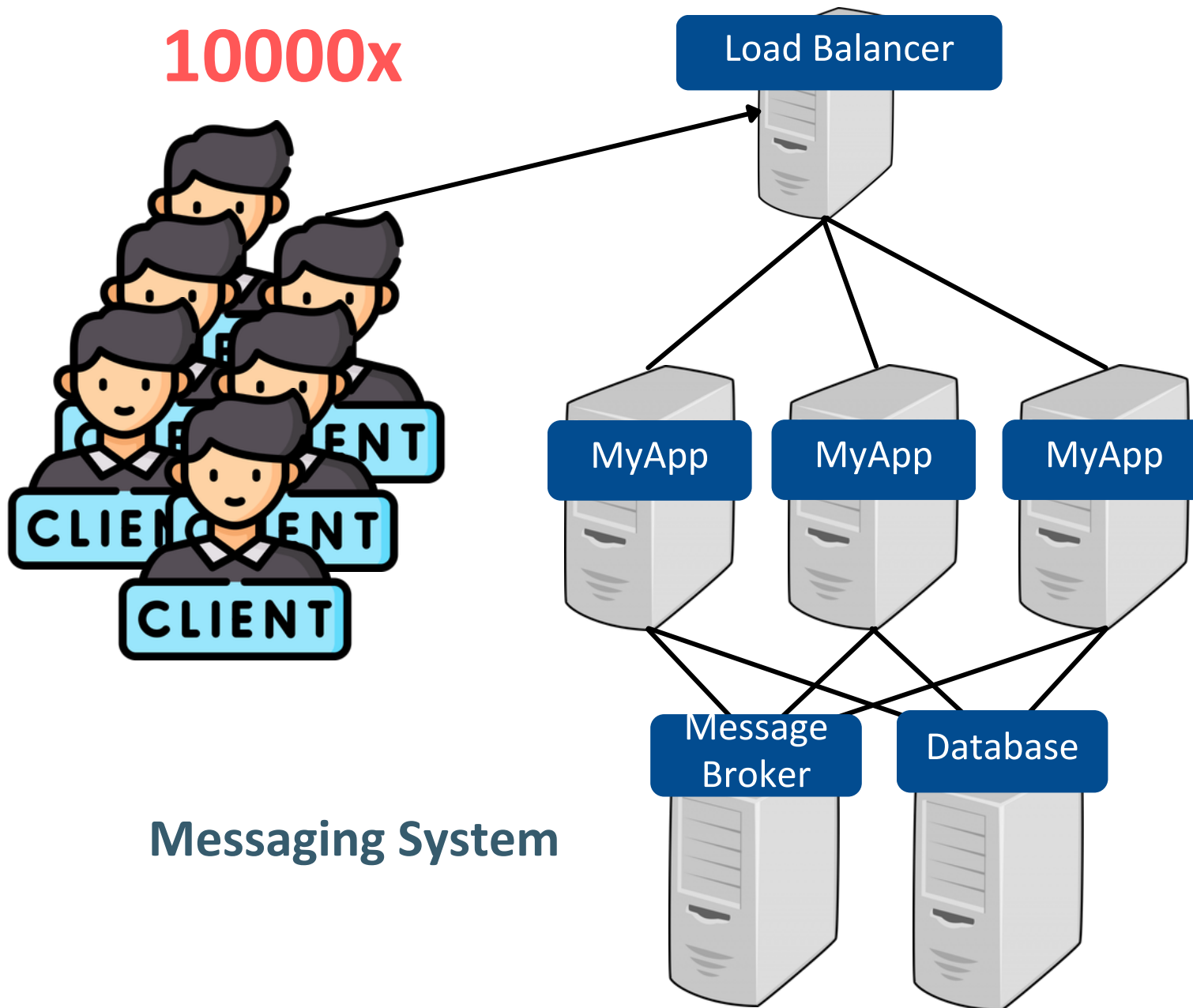
Um case de sucesso

10000x



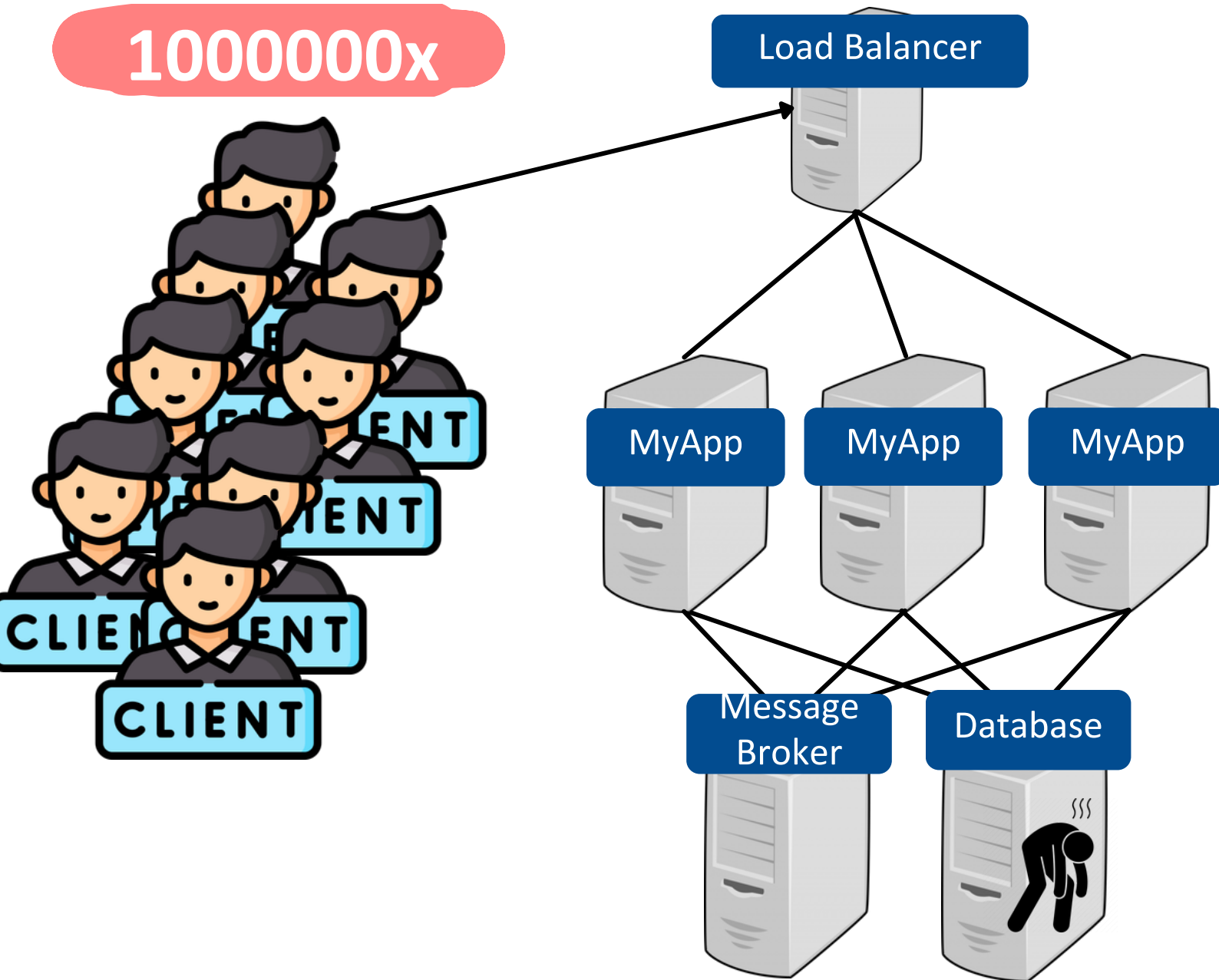
Um case de sucesso

10000x



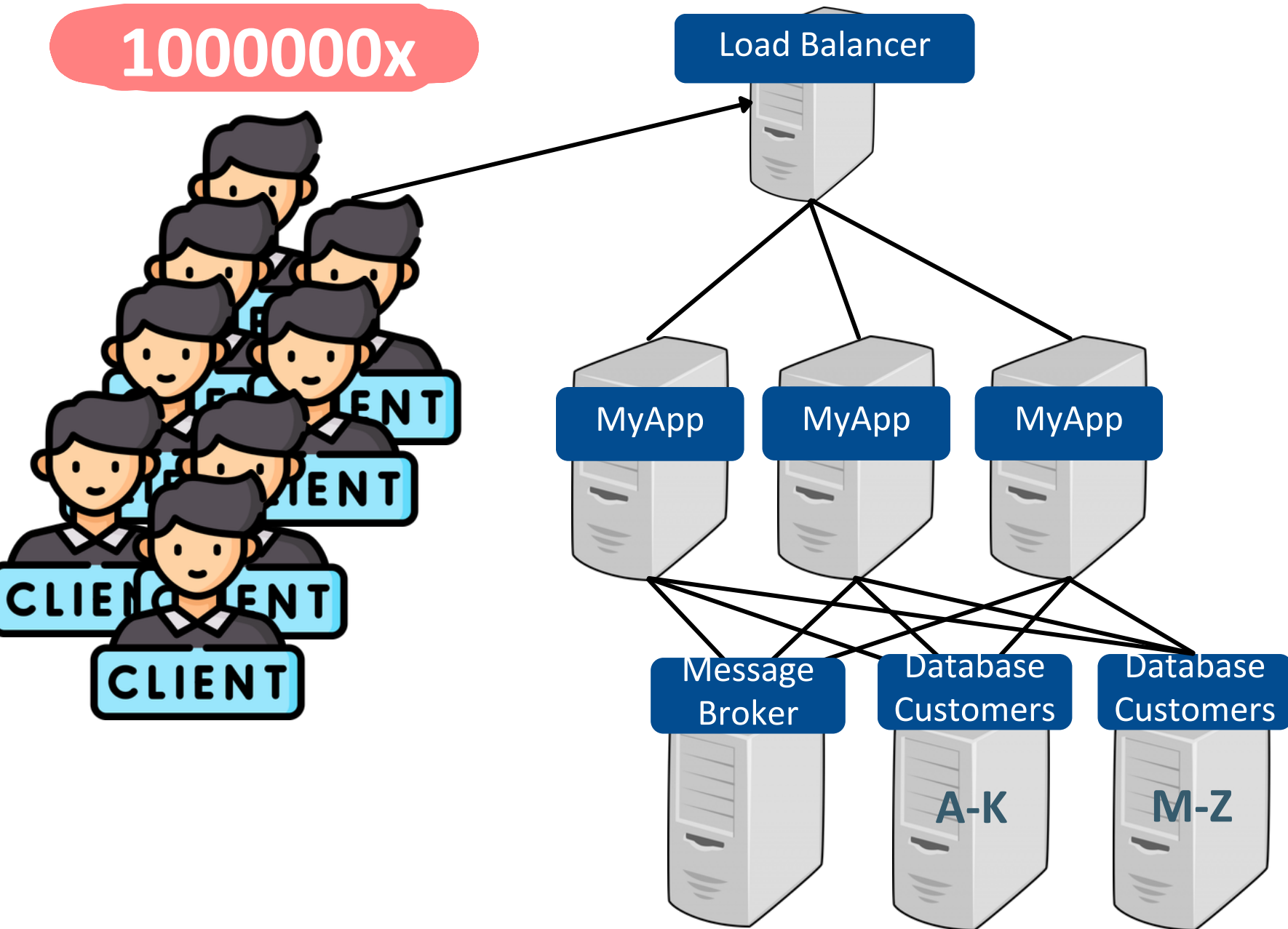
Um case de sucesso

1000000x



Opção A: Sharding

1000000x

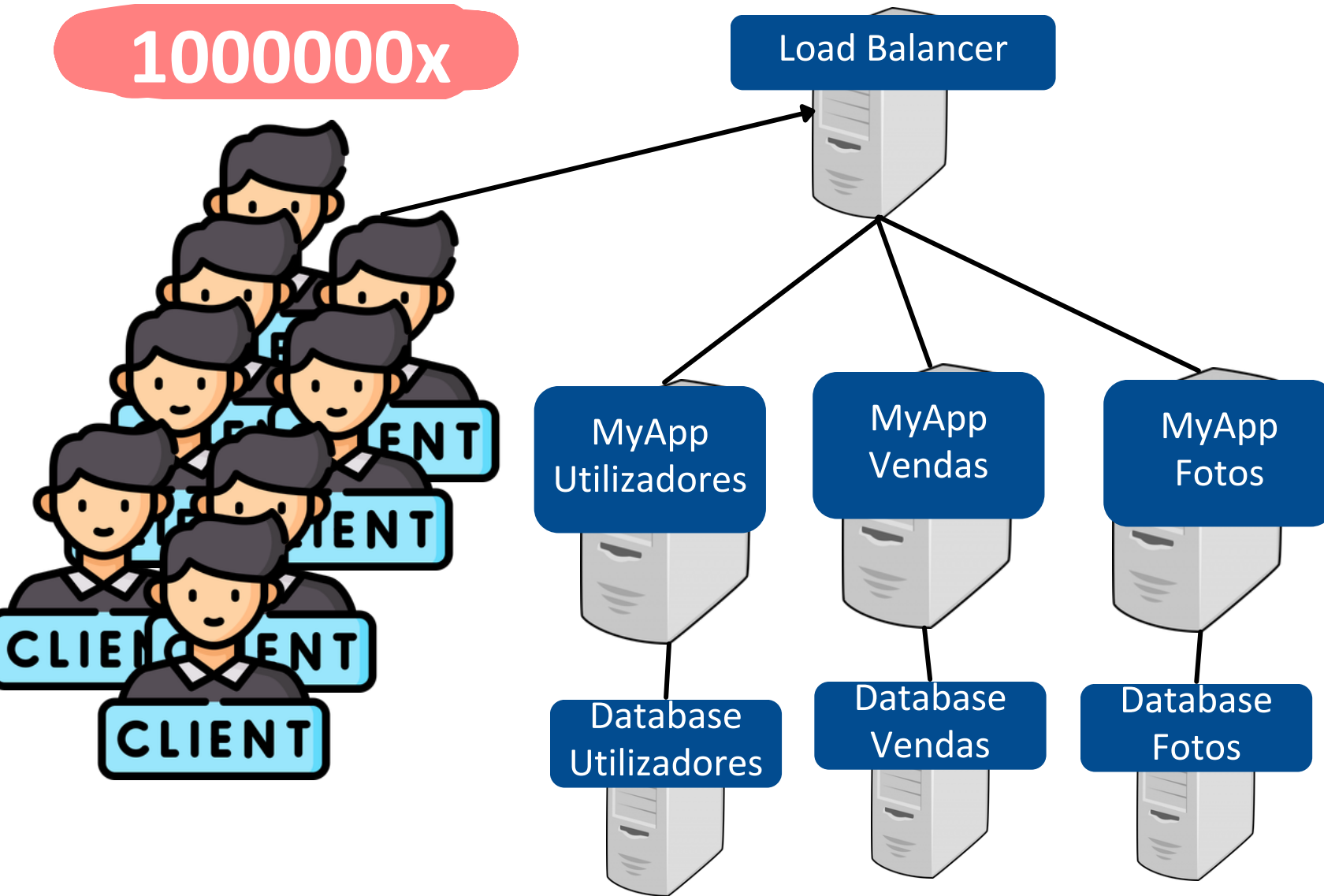


Sharding

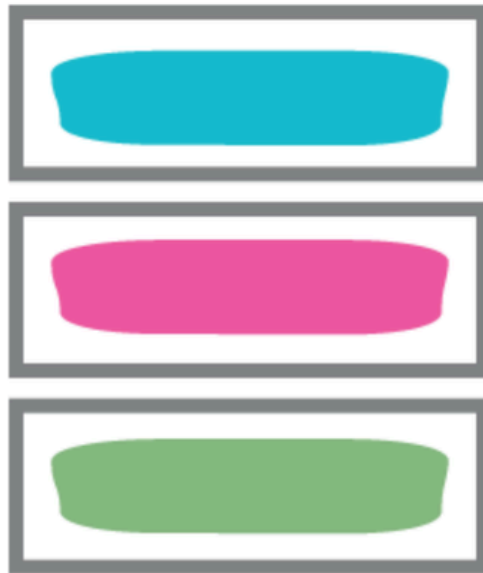
- Dividir toda a base de dados, de acordo com um critério:
 - Utilizadores de 0-4 e de 5-9 (ou outra divisão)
 - Consoante o país (Amazon ES vs Amazon US)
 - Clientes pessoais vs clientes empresariais
- Problema: Toda a base de dados tem de ser partida!
 - Se existir uma transação do cliente 123 (shard1) para o cliente 762 (shard2), em que base de dados fica?
 - **Não funciona!**

Opção B: Micro-serviços

1000000x



Modularização

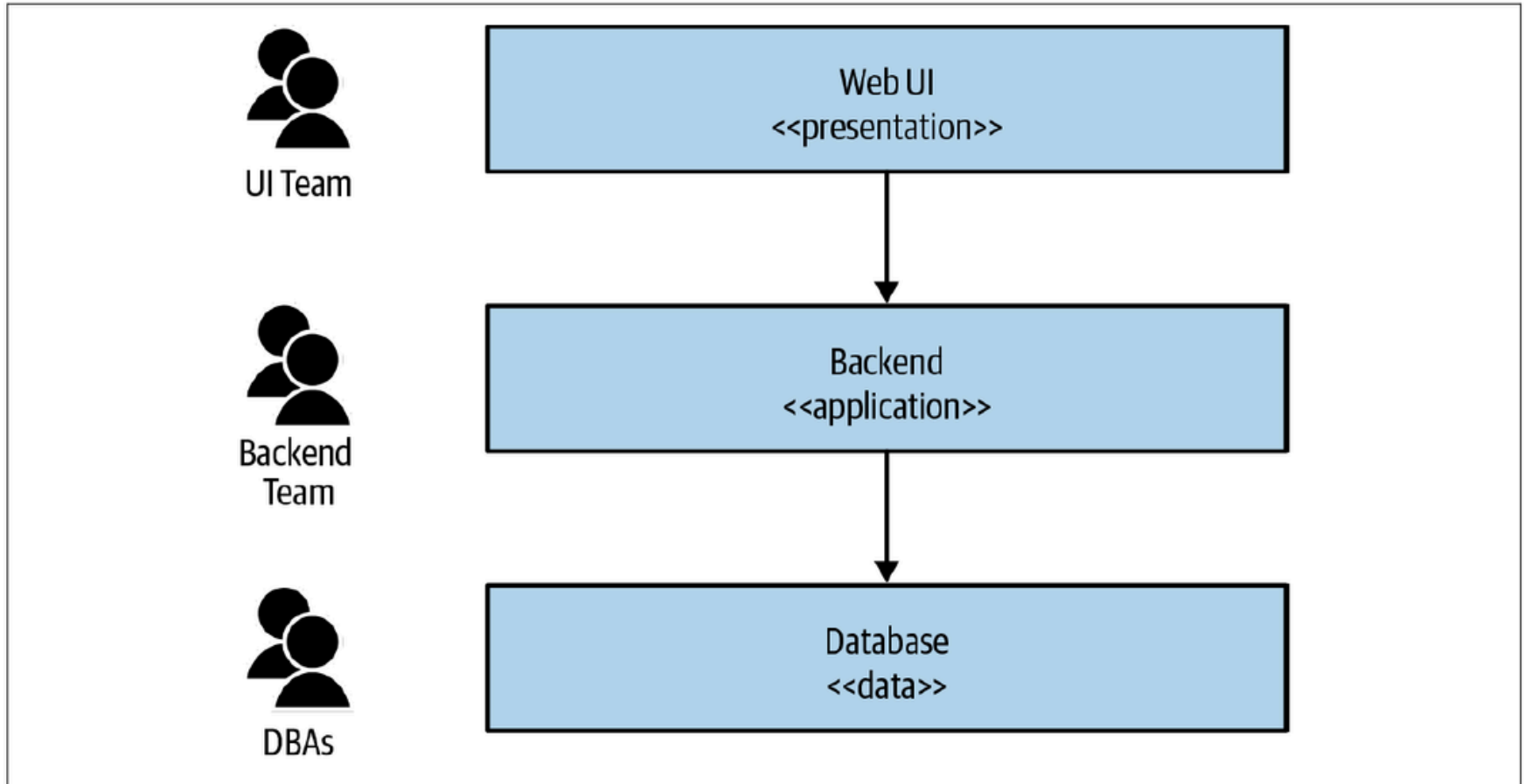


Don't use layers as the top level modules in a complex application...

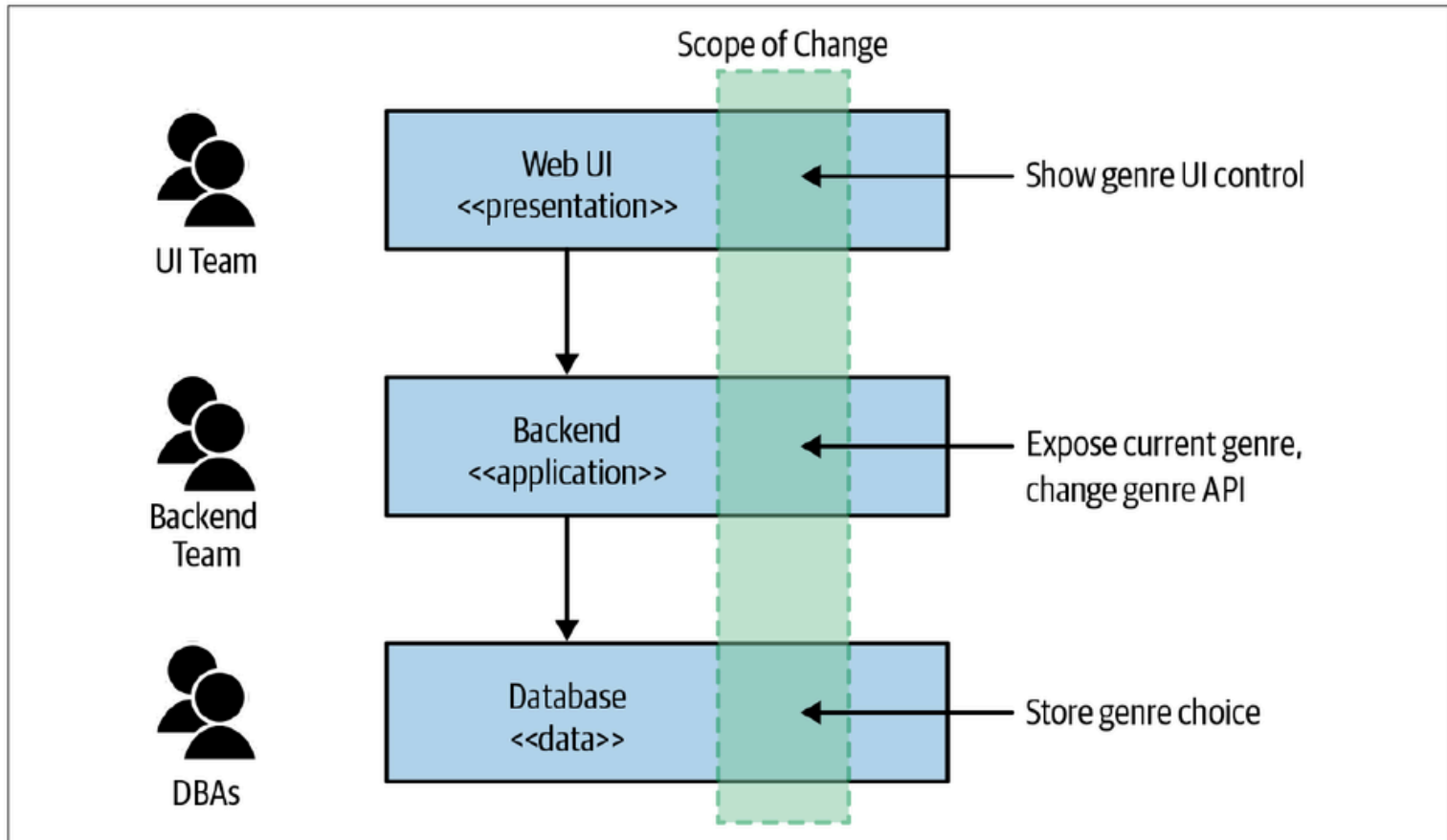


... instead make your top level modules be full-stack

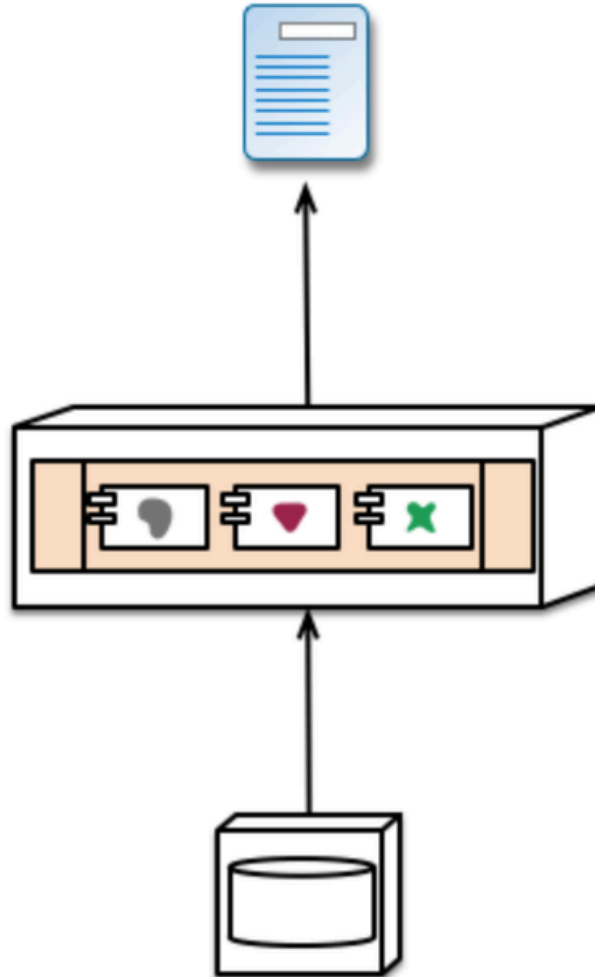
Modularização



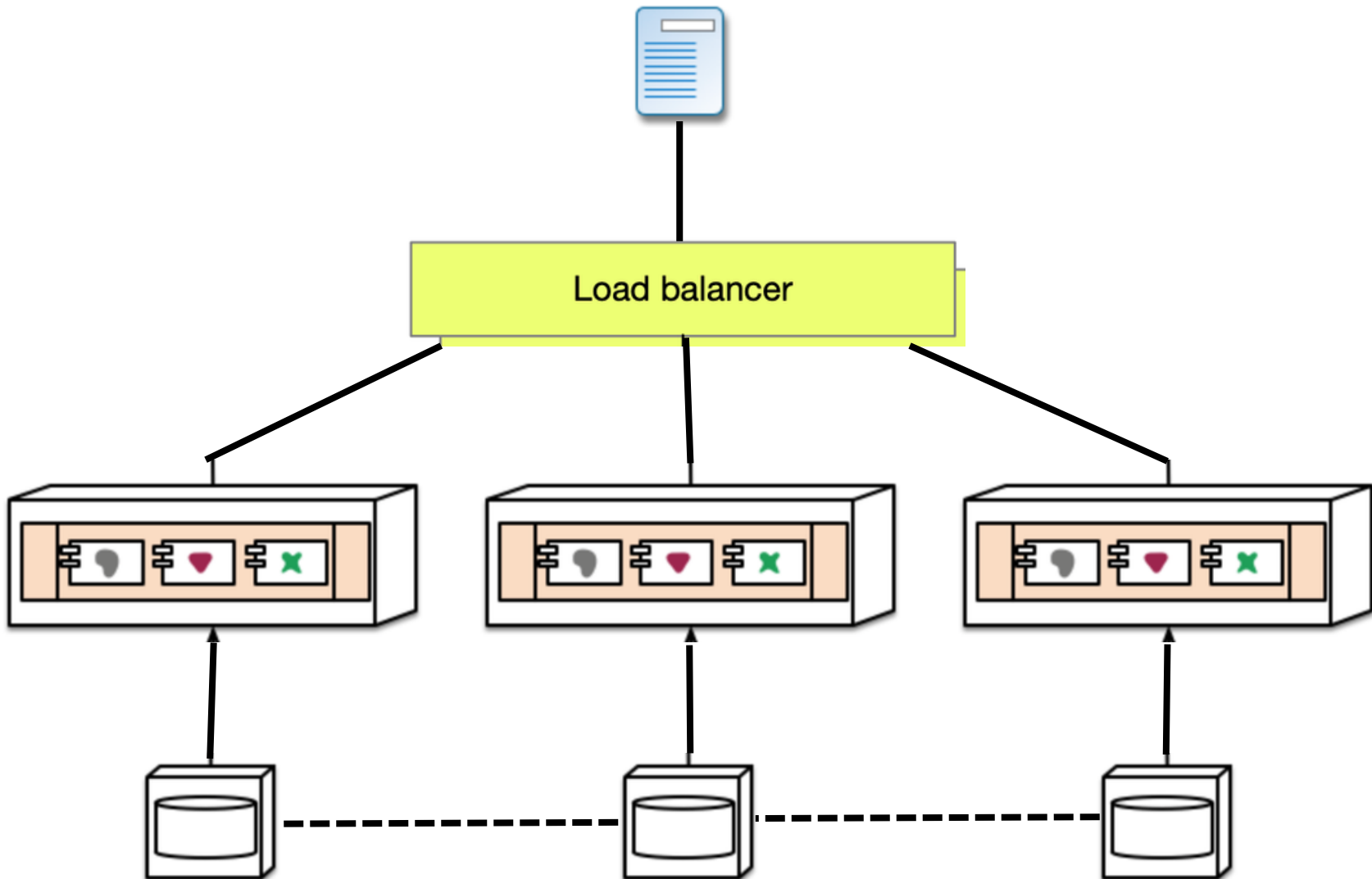
Modularização



Monolithic



Monolithic: Scalling?



Microservices

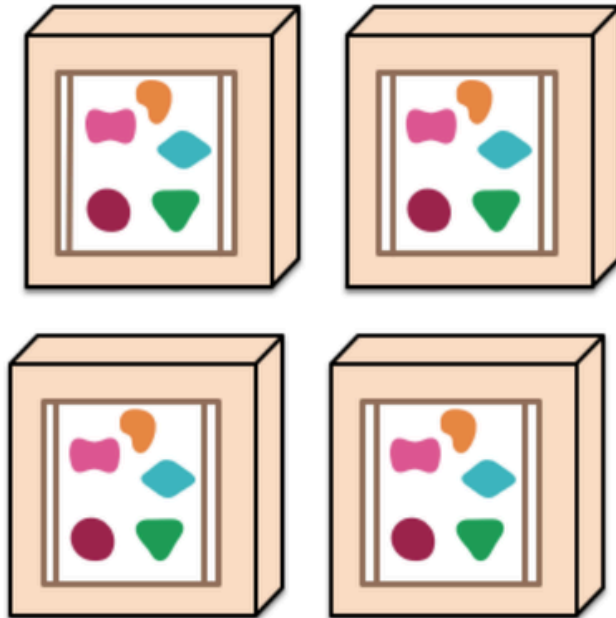
- A aplicação não é desenvolvida como um grande projecto único, mas como um conjunto de projectos mais pequenos que trabalham entre si
- A aplicação não é empacotada como um único artefacto (jar), mas consiste em vários artefactos que funcionam em conjunto para um determinado propósito
- Equipas diferentes podem trabalhar em partes do projecto **independentemente**, desde que concordem em APIs

Monolithic vs MicroServices

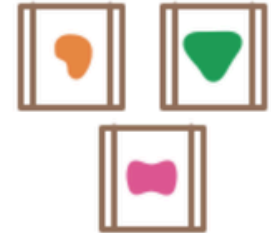
A monolithic application puts all its functionality into a single process...



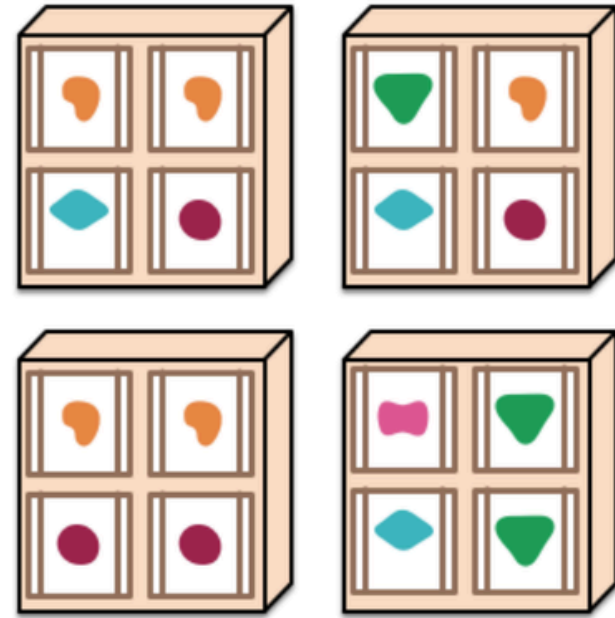
... and scales by replicating the monolith on multiple servers



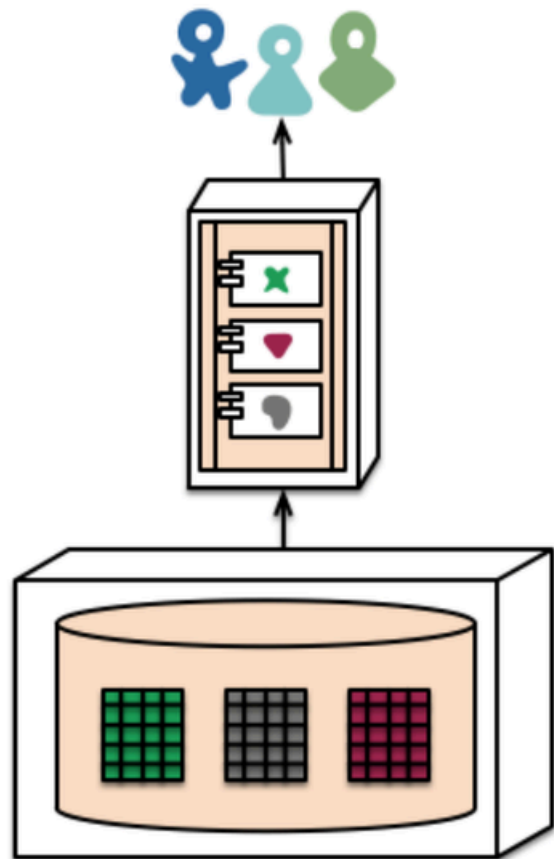
A microservices architecture puts each element of functionality into a separate service...



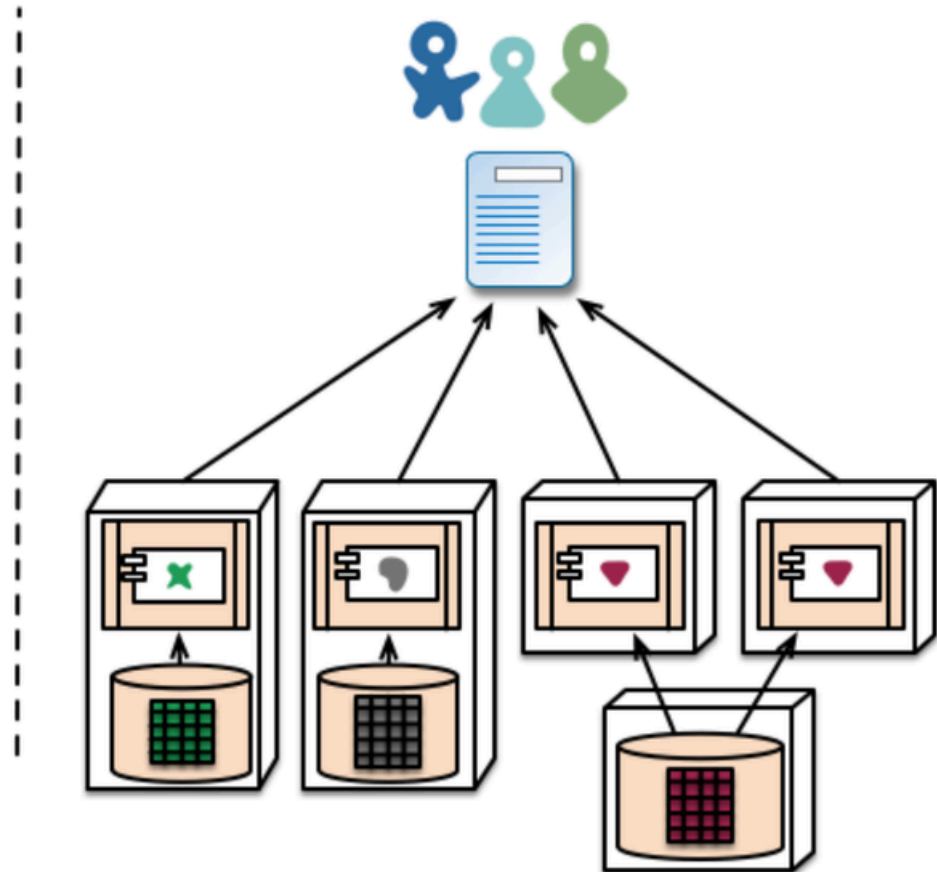
... and scales by distributing these services across servers, replicating as needed.



Monolithic vs MicroServices



monolith - single database



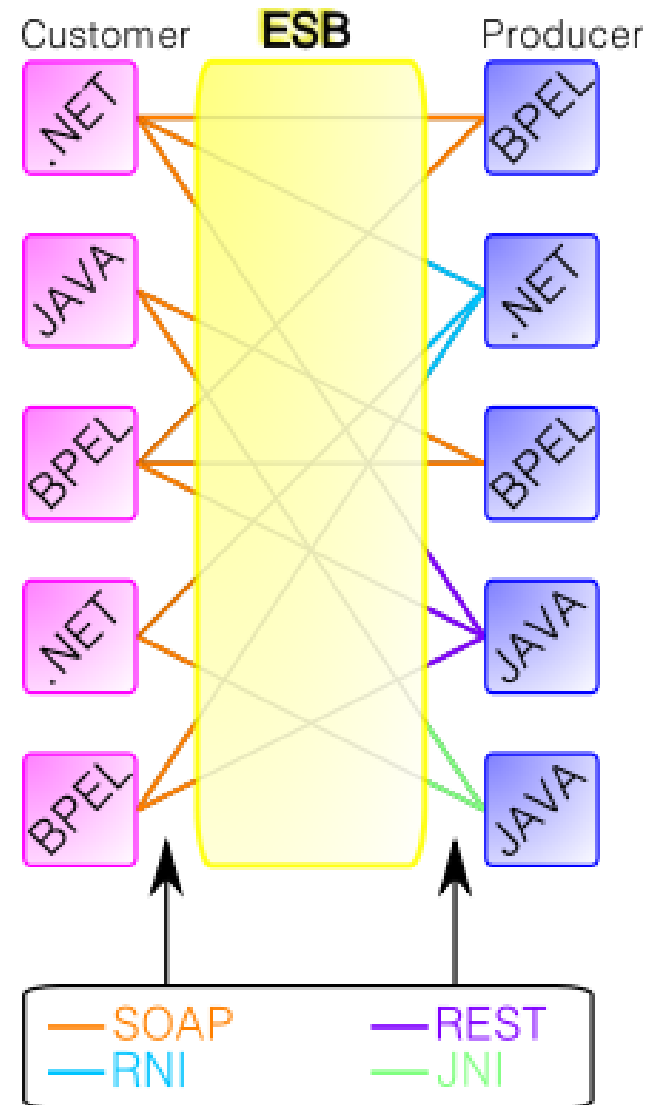
microservices - application databases

Service Oriented Architectures

- A ideia de separar a aplicação em serviços independentes não é nova: O conceito de SOA foi proposto em 1998, e foi bastante adoptado nos anos 2000.
- A comunicação entre serviços era feita directamente por SOAP (REST muito mais recentemente)
- Ou indirectamente por um Enterprise Service Bus
 - Sincronamente: (tipicamente SOAP)
 - Assíncronamente (eventos numa message queue)

Enterprise Service Bus

- Responsável por:
 - Converter formatos de Mensagens (JSON vs XML)
 - Converter protocolos de comunicação (SOAP vs RMI)
 - Monitorizar serviços
 - Monitorizar qualidade de serviço (SLA)
 - Autenticação
 - Autorização
 - Caching
 - ...



SOA vs Microserviços

- Os microserviços evitam esta estrutura no meio (ESB), e comunicam directamente por um protocolo acordado entre os componentes
 - REST, MessageQueue, gRPC
- O conceito de separação de concerns em diferentes artefactos/projectos mantém-se
- A maior diferença é cultural: Os microserviços são mais ágeis, onde cada equipa define a sua API externa, a tecnologia (linguagem, framework, base de dados), e faz o deployment ao seu ritmo
- Microserviços estão associados a Devops.

Modern Enterprise Application Development

<https://www.oreilly.com/ideas/modern-java-ee-design-patterns>

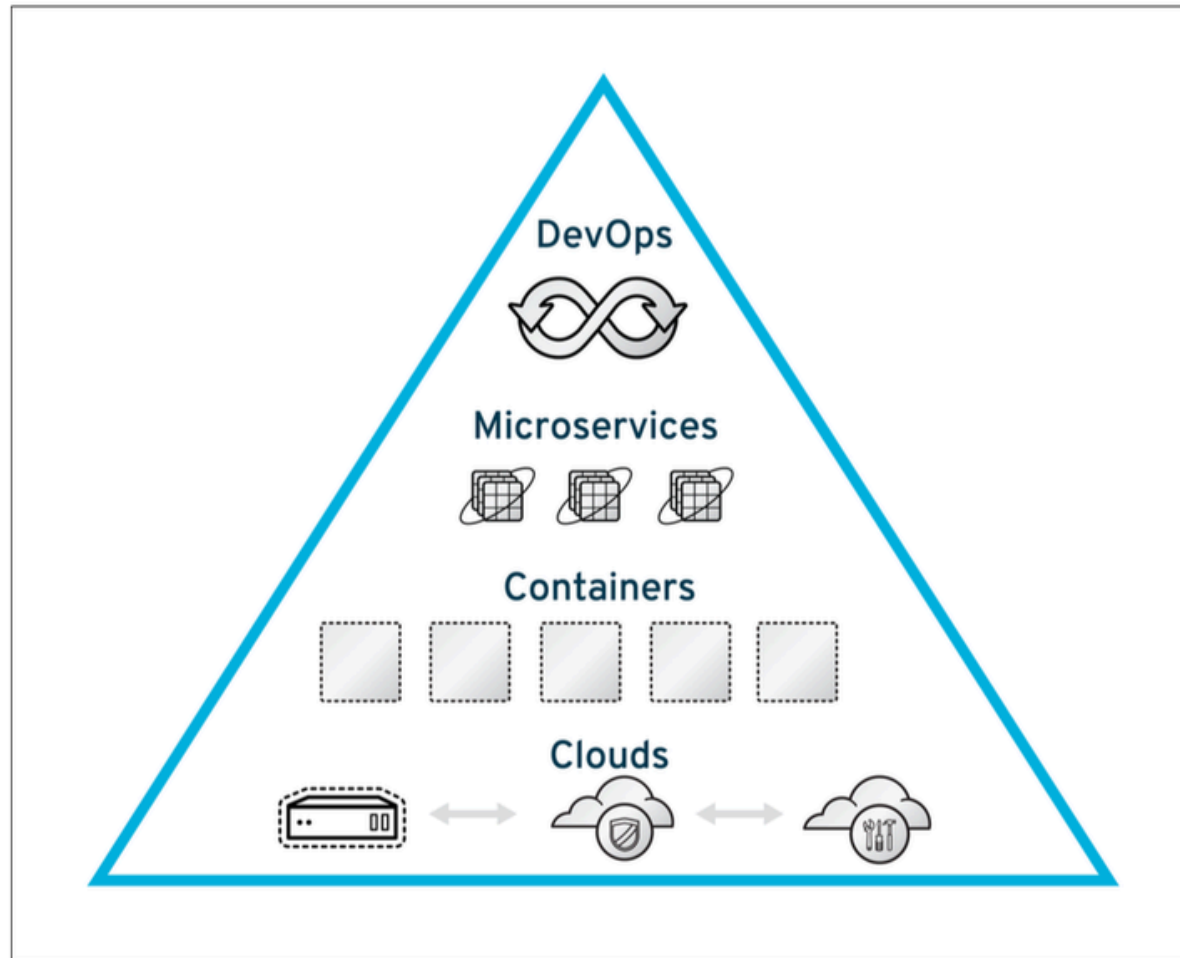
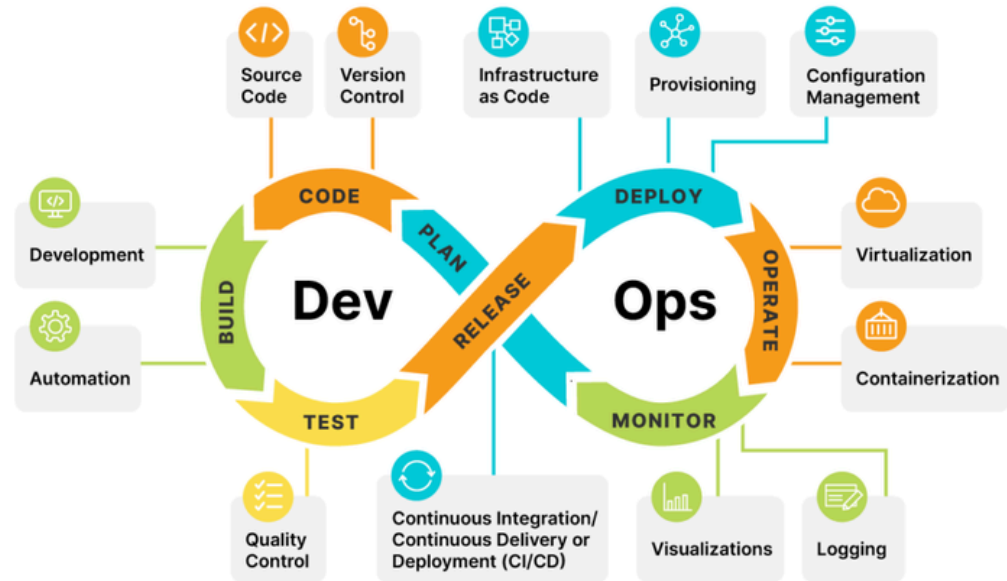


Figure 2-3. The pyramid of modern enterprise application development

DevOps



- Não existe uma equipa de desenvolvimento e uma equipa de operações (responsável por deployment e manutenção dos servidores)
- Os developers são responsáveis por fazer deploy e manter os seus artefactos.
 - Infrastructure as Code (e.g. Docker-Compose)
 - **Continuous Integration** and **Continuous Delivery (CI/CD)**

Vantagens de Microserviços

- Desenvolvimento mais ágil e paralelo:
 - Equipas trabalham mais independentemente
 - Em tecnologias com que estão mais à vontade
 - Mantendo a API, podem mudar o que quiserem (incluindo BDs!)
- Deployment mais eficiente:
 - Cada artefacto é mais pequeno, permitindo escalar a aplicação usando mais máquinas.
- Limites bem definidos entre responsabilidades
 - Se uma funcionalidade está no Microserviço B, sabemos que equipa é responsável por essa funcionalidade.

Desvantagens de Microserviços

- É um sistema (bastante) distribuído:
 - Difícil de programar!
 - Chamadas remotas são lentas
 - Várias falhas possíveis (normalmente comunicação e infraestrutura)
- Pouca garantia de consistência:
 - Numa base de dados de um monólito podemos ter ACID
 - Se uma transação toca em diferentes microserviços, não temos garantias!
 - Normalmente aponta-se para consistência eventual
- Devops implica que todos os developers têm de aprender muito mais sobre infraestrutura

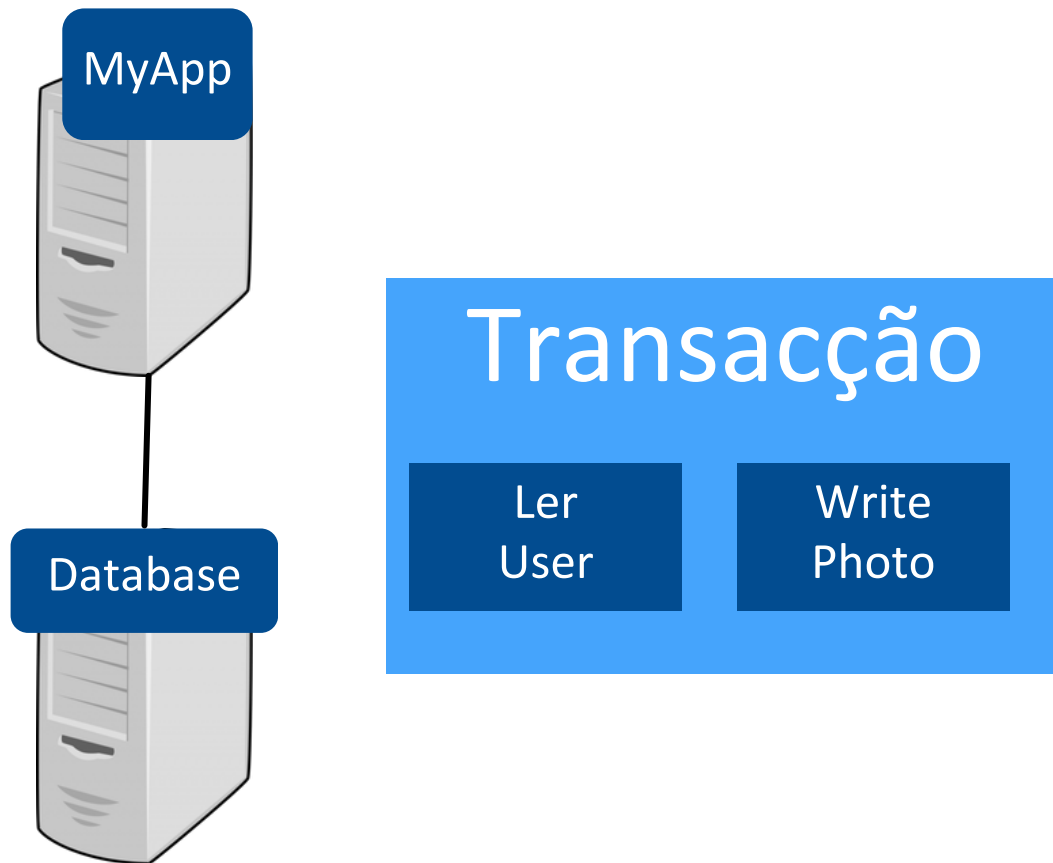


Microservices Are a Tax Your Startup Probably Can't Afford

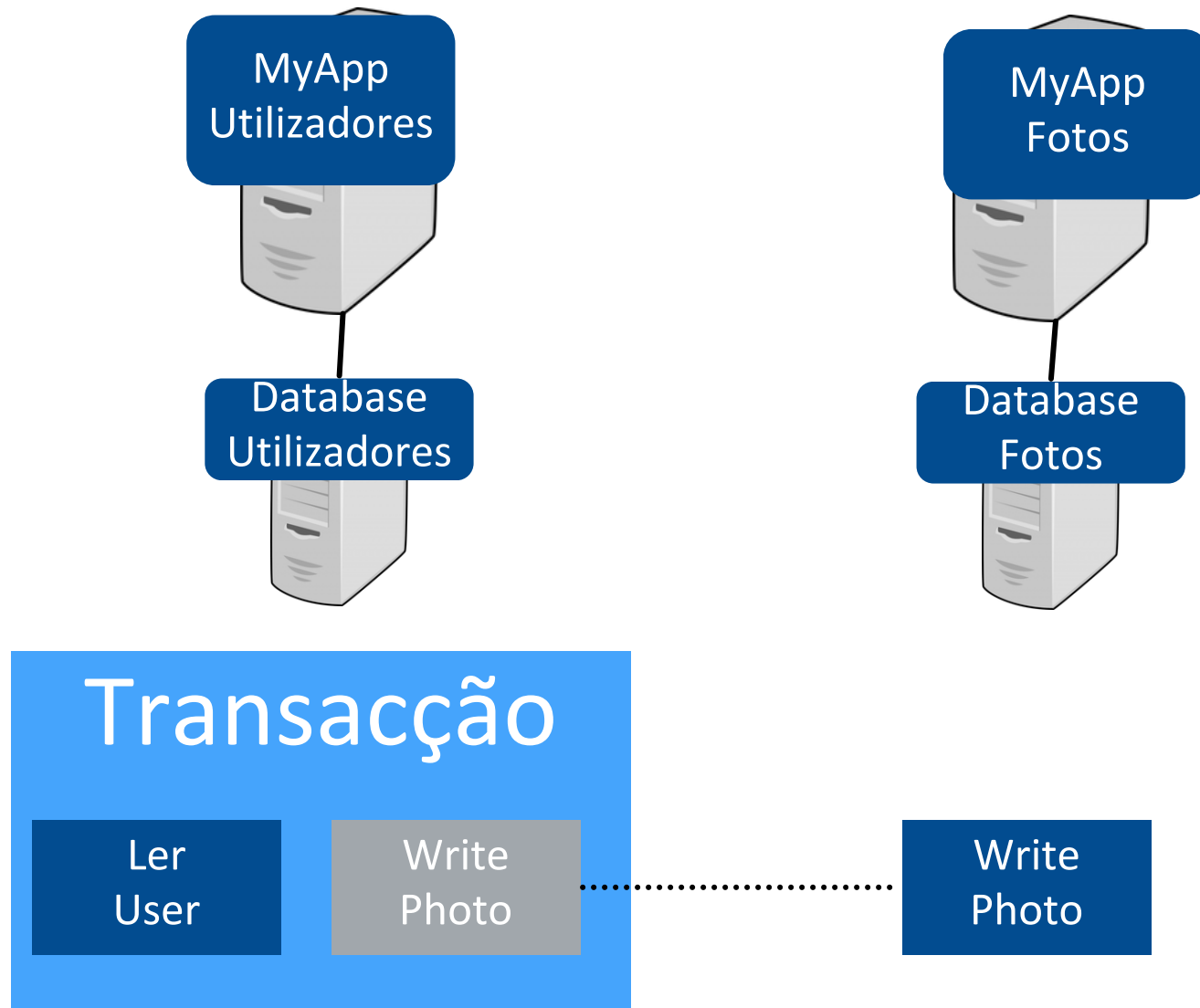
Why splitting your codebase too early can quietly destroy your team's velocity — and what to do instead. In a startup, your survival depends on how quickly you can iterate, ship features, and deliver value to end-users. This is where the foundation...

 oleg pustovit / May 5, 2025

Transacções Distribuídas não têm a mesma consistência



Transacções Distribuídas não têm a mesma consistência



Migração para Microserviços

- A maior parte das aplicações começa o seu desenvolvimento de forma **monolítica**
- Quando migrar?

Migração para Microserviços

- A maior parte das aplicações começa o seu desenvolvimento de forma **monolítica**
- Quando migrar?
 - Autonomia das equipas
 - Carga de trabalho aumentando
 - Robustez
 - Crescimento do número de desenvolvedores
 - Novas tecnologias

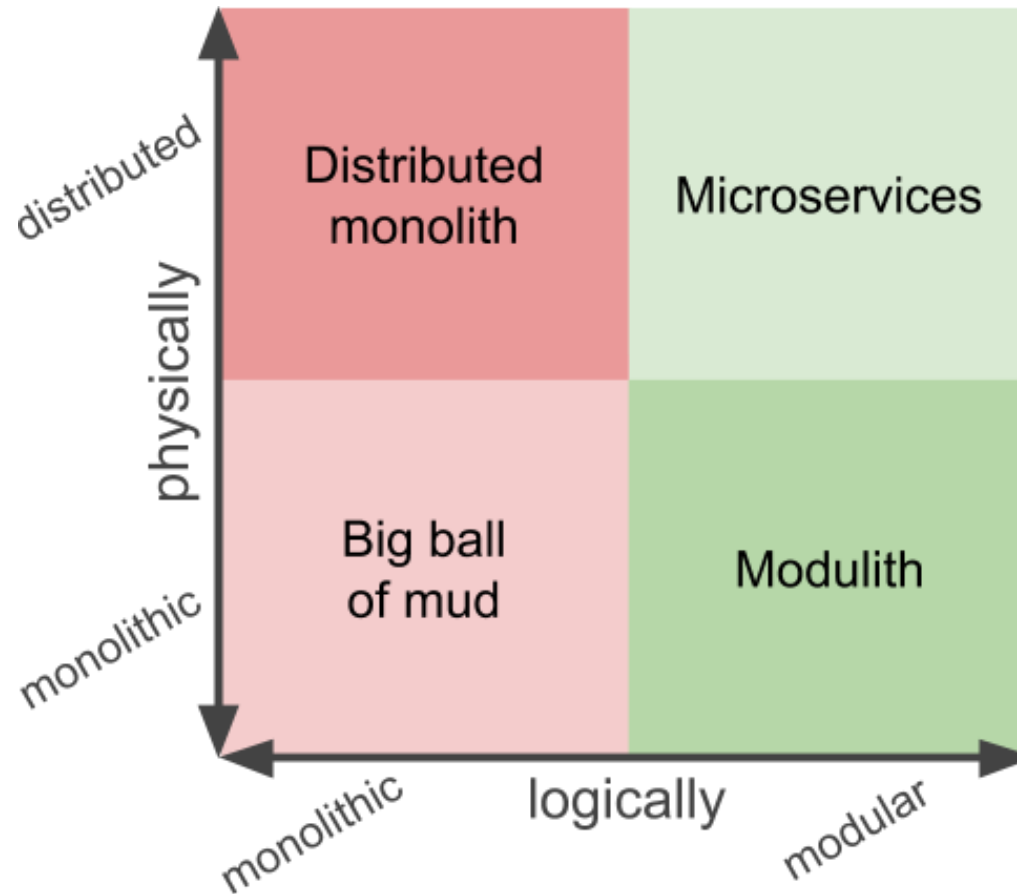
Migração para Microserviços

- A maior parte das aplicações começa o seu desenvolvimento de forma **monolítica**
- Quando migrar?
 - Autonomia das equipas
 - Carga de trabalho aumentando
 - Robustez
 - Crescimento do número de desenvolvedores
 - Novas tecnologias
- Quando **não** migrar?

Migração para Microserviços

- A maior parte das aplicações começa o seu desenvolvimento de forma **monolítica**
- Quando migrar?
 - Autonomia das equipas
 - Carga de trabalho aumentando
 - Robustez
 - Crescimento do número de desenvolvedores
 - Novas tecnologias
- Quando **não** migrar?
 - Domínio incerto
 - Startups
 - Software instalado e gerenciado pelo cliente
 - **Não ter um bom motivo!**

Tipos de Monólito



Tipos de Monólito

- **Monólito Modular**
 - **Alta coesão** dentro de módulos com as **interfaces bem definidas**, mais **fácil de manter**. Modelo ideal de Monólito

Tipos de Monólito

- **Monólito Modular**

- **Alta coesão** dentro de módulos com as **interfaces bem definidas**, mais **fácil de manter**. Modelo ideal de Monólito

- **Monólito Distribuído**

- Uso de tecnologias distribuídas (lifecycle monolítico)
 - Complexidade distribuída + limitações monolíticas

Tipos de Monólito

- **Monólito Modular**

- **Alta coesão** dentro de módulos com as **interfaces bem definidas**, mais **fácil de manter**. Modelo ideal de Monólito

- **Monólito Distribuído**

- Uso de tecnologias distribuídas (lifecycle monolítico)
 - Complexidade distribuída + limitações monolíticas

- **Big Ball of Mud**

- **Difícil** de entender e modificar. Mudanças têm um **alto risco de efeitos colaterais**.
 - Há uma complexidade imensa em acrescentar novas features, corrigir bugs e mesmo em manter o sistema funcional.

Tipos de Monólito

- **Monólito de Camadas**

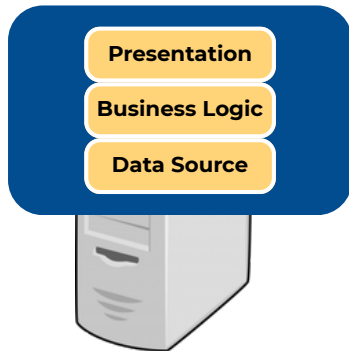
- Estrutura em **camadas** (e.g., MVC). Torna mais fácil a organização do código
 - Problemas com escalabilidade, acoplamento e complexidade.
 - Desenvolvido baseado em **separação técnica e não de negócio**.

- **Monólito com componentes reutilizáveis**

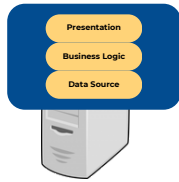
- Faz uso de **bibliotecas e componentes compartilhados**.
 - Promove reutilização de código.
 - Ainda tem problemas sérios com escalabilidade, flexibilidade e acoplamento.

Migração

Monólito → Monólito Modular → Microserviços



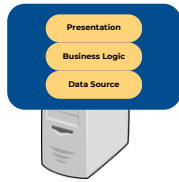
Migração



Monólito → Monólito Modular → Microserviços

Aproveitar o código ou criar funcionalidades novas?

Migração



Monólito → Monólito Modular → Microserviços

Deve-se aproveitar o máximo de funcionalidade possível.
Nos casos onde não é possível, deve-se reimplementar.

- Reimplementação sempre é mais cara (\$\$\$ e mão de obra), mas por muitas vezes é a única solução viável.

Avaliação da migração

- **Checkpoints**
 - Verifique se os microserviços continuam a fazer sentido para o negócio
- Dados **qualitativos** e **quantitativos**
 - **Feedback** e **monitorização**
- *Sunken Cost Fallacy*
 - **Mudar de estratégia** se necessário

Monólitos vs Micro-Serviços